DECA Lab Logbook

Part 2 | Section 2: State Machines

Section 2: State Machines

Aim

To design and implement a finite state machine (FSM) for a vending machine controller in Issie, deriving next-state Boolean logic from a state transition diagram, adding output combinational logic, and extending the design with a vending-complete sensor.

Before the Lab — Vending Machine State Diagram

The vending machine has two 1-bit inputs: C (coin inserted) and S (product selected). The 2-bit state register encodes four states:

State code	State name	Description
00	START	Initial state — waiting for any input
01	W/SEL	Coin inserted, waiting for selection
10	W/COIN	Selection made, waiting for coin
11	VEND	Dispense product

Next-State Truth Table

Using the state transition diagram, the full truth table for current state (A, B = MSB, LSB) and inputs C, S gives the next state (A', B'):

A B (current)	C	S	A' B' (next)
00	0	0	00
00	0	1	01
00	1	0	10
00	1	1	11
01	0	0	00
01	0	1	01
01	1	0	10
01	1	1	11
10	0	0	00
10	0	1	01
10	1	0	10
10	1	1	11

A B (current)	C	S	A' B' (next)
11	0	0	00
11	0	1	00
11	1	0	00
11	1	1	00

Karnaugh Map Simplification

Two Karnaugh maps are derived — one for LSB (B') and one for MSB (A') — to find minimal Boolean expressions.

LSB Karnaugh map gives: $B' = A\bar{B} + S\bar{B}$

MSB Karnaugh map gives: $A' = CA + AB$

(Where A and B are the current-state MSB and LSB respectively.)

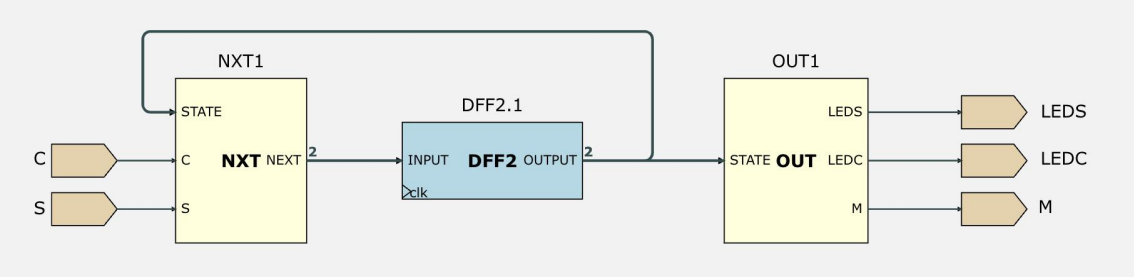
State Machine Implementation

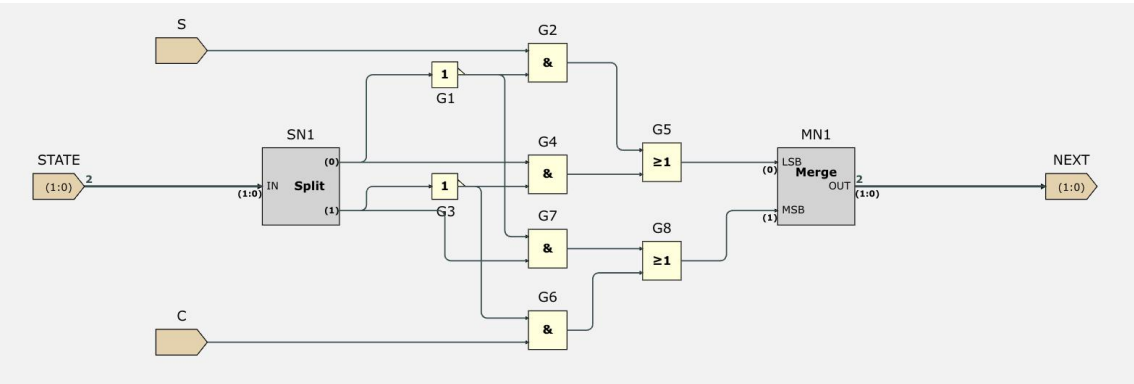
Aim

Implement the vending machine FSM in Issie using a hierarchical design with separate sheets for the register, next-state logic, output logic, and top-level wiring.

Sheets

- DFF2: 2-bit D-type register storing the current state
- NXT: combinational logic computing next state (A', B') from current state (A, B) and inputs C, S
- OUT: combinational logic generating outputs M, LEDS, LEDC from current state
- VEND: top-level sheet connecting DFF2, NXT and OUT





How It Works

On each rising clock edge, DFF2 latches the output of NXT as the new current state. The NXT block implements the Boolean expressions $B' = A\bar{B} + S\bar{B}$ and $A' = CA + AB$ using logic gates derived from the Karnaugh maps. The OUT block decodes the current state into the three output signals.

Output Logic (OUT block)

Output Truth Table

State	Code A B	M (motor)	LEDS (selection LED)	LEDC (coin LED)
START	00	0	1	1
W/SEL	01	0	1	0
W/COIN	10	0	0	1
VEND	11	1	0	0

Boolean Expressions for Outputs

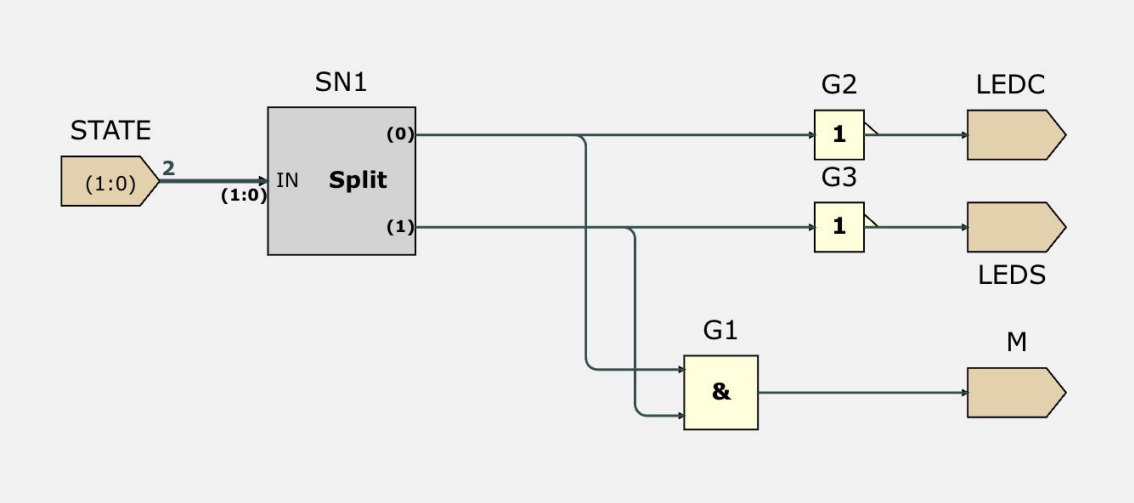
$LEDC = \bar{B}$ (lit when LSB = 0, i.e. START or W/COIN)

$LEDS = \bar{A}$ (lit when MSB = 0, i.e. START or W/SEL)

$M = AB$ (motor on only in VEND state, both bits = 1)

OUT Implementation

Each bit of the state bus is passed through a Bus Compare (equal to 0) to detect when that bit is 0, driving the corresponding LED. An AND gate of both state bits drives the motor output M.



Section 2 Challenge: Extending the Vending Machine

Aim

Add a 1-bit vending-complete sensor input V to the FSM. When V = 0 the product is still being dispensed and the machine must stay in the VEND state (11). When V = 1, vending is complete and the machine returns to START (00).

Extended Next-State Truth Table (with V)

The VEND state (11) now has two possible transitions depending on V. All other state transitions are independent of V (represented as X — don't care).

A B (current)	C	S	V	A' B' (next)
00	0	0	X	00
00	0	1	X	01
00	1	0	X	10
00	1	1	X	11
01	0	X	X	01
01	1	X	X	11
10	X	0	X	10
10	X	1	X	11
11	X	X	0	11
11	X	X	1	00

Derived Boolean Expressions (with V)

From the extended truth table, the next-state expressions become:

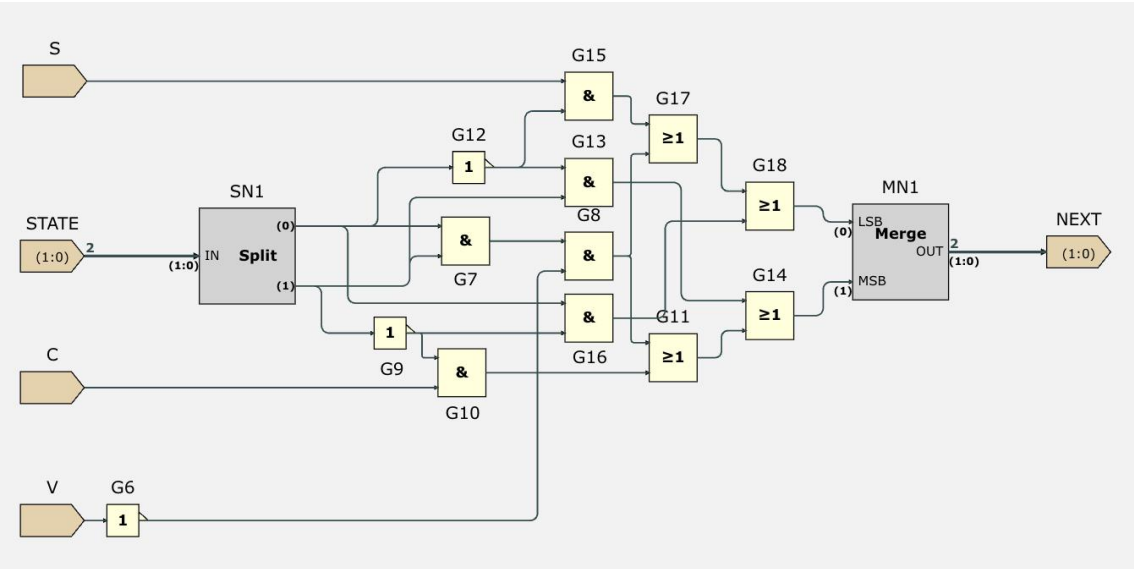
Cnext = C̄inC + CinS̄in + CinSinV̄

Snext = S̄inS + SinC̄in + CinSinV̄

Implementation in NXT Block

The additional gates required in NXT implement the extra term C

The new input V connects to a NOT gate (V̄). The AND gate combining C



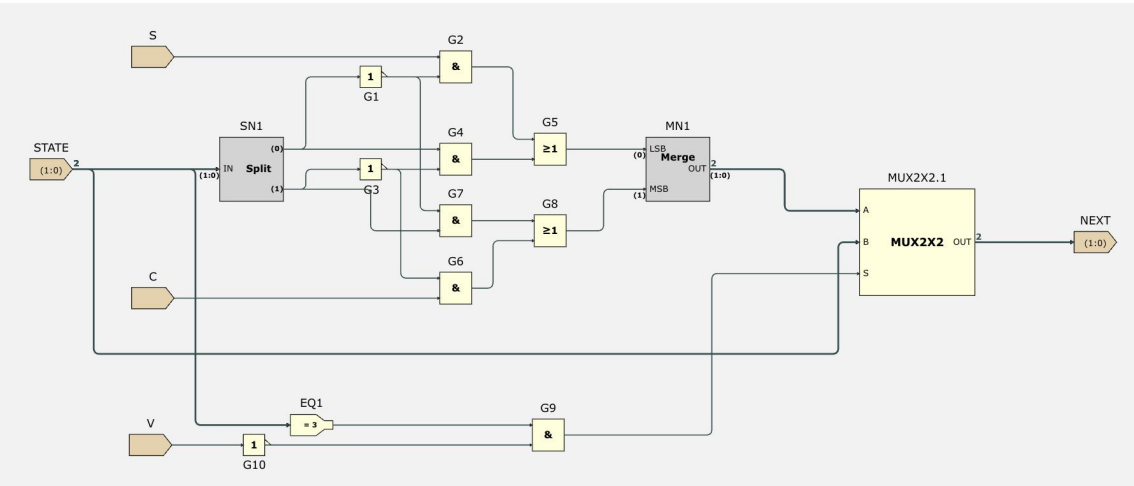
Alternative Implementation (MUX-based)

Motivation

Deriving and simplifying the full Boolean expressions with three input variables is complex. An alternative approach uses the existing NXT logic unchanged and adds a MUX to intercept the VEND-state output.

How It Works

- A Bus Compare checks if the current state is 11 (VEND). When true, it outputs 1.
- A NOT gate inverts V: NOT(V) = 1 while vending is in progress.
- An AND gate combines the comparator output and NOT(V) — outputs 1 only when in VEND state AND V = 0.
- A MUX2X2 selects between the NXT output and the current state (11) based on this AND signal.



When V = 0 and state = 11: the AND gate output = 1, so MUX selects the current state 11 — the machine stays in VEND.

When V = 1 and state = 11: NOT(V) = 0, AND output = 0, MUX selects the NXT output — state transitions to 00 (START) as normal.

All other states: comparator output = 0, AND = 0, MUX passes the NXT output unchanged.

No changes to DFF2 or OUT are needed. Only a NOT gate, a Bus Compare, an AND gate, and a 2-bit MUX are added.